

Sprawozdanie z realizacji projektu - Język Java

Markiiian Novytskyi, Arseni Fayeue

16 czerwca 2026

1. Tytuł projektu

Java Royale: Multiplayer Top-Down Battle Royale Shooter

Wielosobowa gra akcji typu battle royale z widokiem z góry. Architektura klient-serwer z autorytatywnym serwerem gry, klientem graficznym libGDX oraz warstwą sieciową Netty (TCP dla lobby, UDP dla rozgrywki).

2. Założone vs. zrealizowane funkcjonalności

Funkcjonalność	Założona	Zrealizowana
System lobby (połączenie, roster, ready, countdown)	Tak	Tak
Mechanika ruchu (WSAD + predykcja klienta)	Tak	Tak
System walki (strzelanie, trafienia, eliminacja)	Tak	Tak
Synchronizacja świata (snapshots UDP)	Tak	Tak
Mechanika strefy (storm, obrażenia poza strefą)	Tak	Tak
Warunki zwycięstwa (match end, statystyki)	Tak	Tak
Skalowalność do 25–50 graczy (architektura)	Tak	Częściowo
Testy obciążeniowe wielu klientów	Tak	Nie

Zrealizowano: 6 z 7 zaplanowanych funkcjonalności (86%)

3. Realizacja wymagań

Funkcjonalne vs. niefunkcjonalne

Typ Wymagań	Założone	Zrealizowane
-------------	----------	--------------

Funkcjonalne	7	6
Niefunkcjonalne	5	4

Procent realizacji: Funkcjonalne: 86%, Niefunkcjonalne: 80%

Odchylenie od specyfikacji początkowej

W pierwotnej specyfikacji cała komunikacja sieciowa opisana była jako TCP. W implementacji zastosowano model hybrydowy:

- **TCP (port 25565)** — lobby: handshake, roster, matchmaking, countdown, koniec meczu.
- **UDP (port 25566)** — rozgrywka: input gracza, snapshots świata, strefa, przedmioty.

Uzasadnienie: niskie opóźnienia i wysoka częstotliwość pakietów w fazie gry; zdarzenia krytyczne (śmierć, koniec meczu) nadal przesyłane przez TCP.

Wymagania niefunkcjonalne

- **Wydajność (60 FPS)** — Tak (libGDX, płynna rozgrywka dla wielu klientów testowych).
- **Skalowalność (25–50 graczy)** — Częściowo (architektura tick-based; testy głównie dla 2–4 graczy).
- **Niskie opóźnienia** — Tak (UDP dla gameplay, predykcja i rekonsylacja po stronie klienta).
- **Stabilność** — Częściowo (obsługa rozłączeń, walidacja seq; brak reconnect w trakcie meczu).
- **Modularność Maven** — Tak (moduły `common`, `networking`, `app`).

4. Rzeczywisty nakład pracy

- Planowany czas pracy: 155 godzin
- Rzeczywisty czas pracy: 155 godzin

Rozbicie (zgodnie ze specyfikacją):

- Architektura i Maven: 10 h
- Networking (Netty TCP/UDP): 30 h
- Rendering (libGDX): 45 h
- Logika gry (kolizje, strefa, strzelanie): 30 h
- UI / HUD: 15 h
- Testowanie i debugging: 25 h

5. Napotkane problemy i rozwiązania

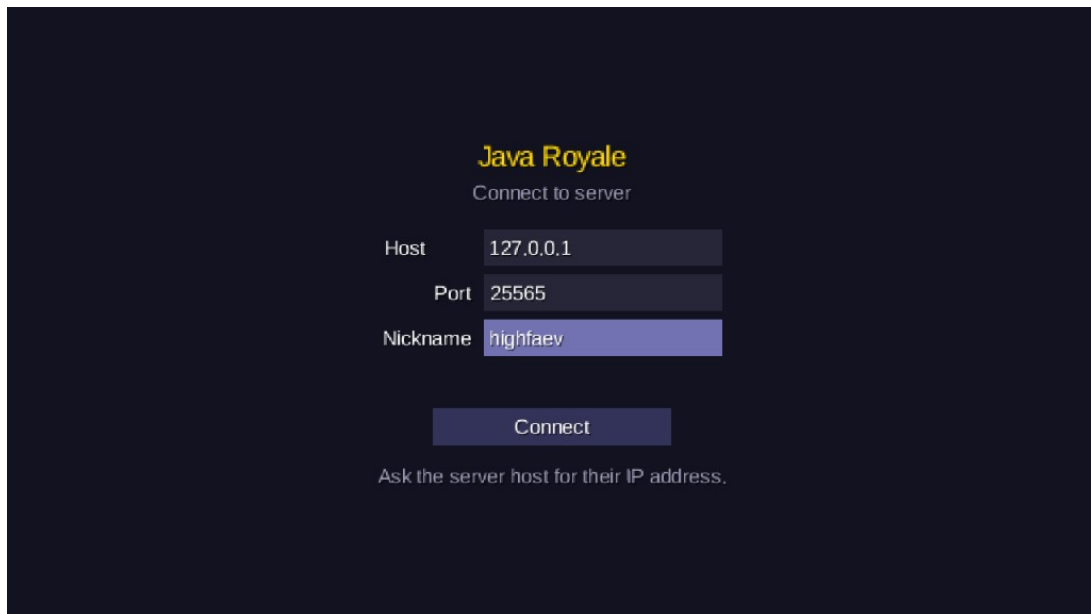
- **Regresja dekodowania rosteru TCP:** częściowe ramki TCP powodowały pustą listę graczy w lobby.
Rozwiązanie: test regresyjny `TcpFromServerDecoderRosterTest` oraz bezpieczne parsowanie w `TcpFromServerDecoder`.
- **ClassNotFoundException przy uruchomieniu serwera:** brak kompilacji przed `mvn exec:java`.
Rozwiązanie: `mvn clean install` przed startem serwera/klienta.
- **Odchylenie TCP vs UDP:** specyfikacja zakładała TCP; gameplay wymaga niższych lagów.
Rozwiązanie: hybryda TCP (lobby) + UDP (gra), opisana w `networc_structure.md`.
- **Synchronizacja pozycji:** przeskoki przy opóźnieniu sieci.
Rozwiązanie: predykcja po stronie klienta, autorytatywna symulacja serwera (`MatchRuntime`), interpolacja (LERP) dla innych graczy.
- **Testy jednostkowe warstwy sieciowej:** niskie pokrycie na początku fazy testów.
Rozwiązanie: rozszerzenie testów JUnit 5 dla `Wire`, `UdpHeader`, dekodowników TCP oraz logiki UDP (`MatchRuntime`, `ZoneManager`, `WorldSnapshotCodec`).

6. Możliwości Rozwoju

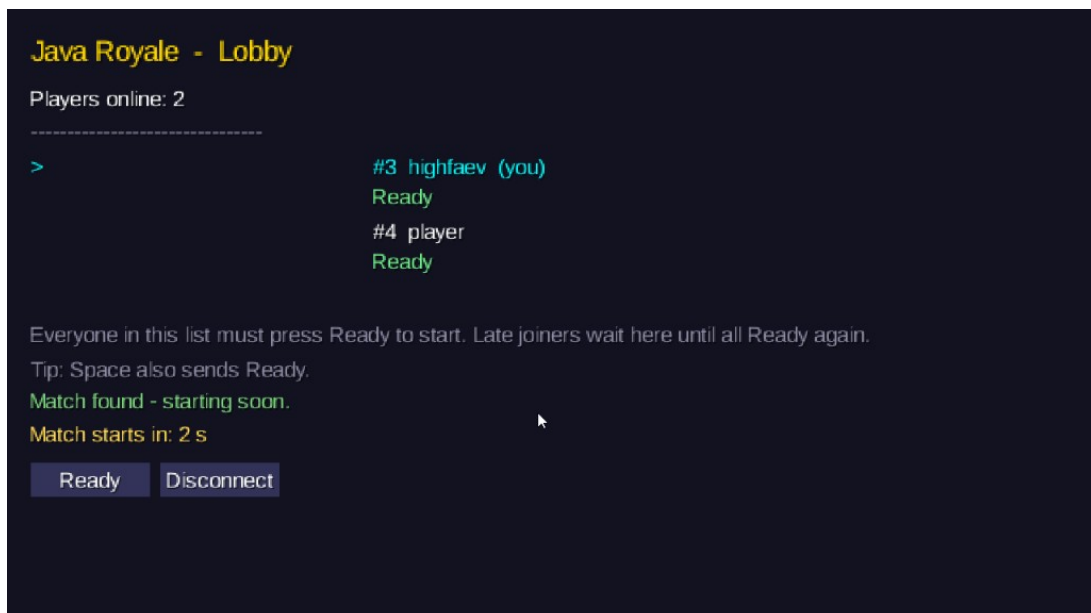
- Delta compression snapshotów i interest management (mniej ruchu przy 25+ graczach).
- Reconnect w trakcie meczu (zachowanie stanu po rozłączeniu TCP).
- Testy obciążeniowe (stress test wielu klientów na jednej maszynie).
- Integracja JaCoCo w CI dla raportu pokrycia kodu.
- Tryb drużynowy, czat głosowy / tekstowy.

7. Zrzuty Ekranu

Poniżej zamieszczono zrzuty ekranu aplikacji Java Royale.



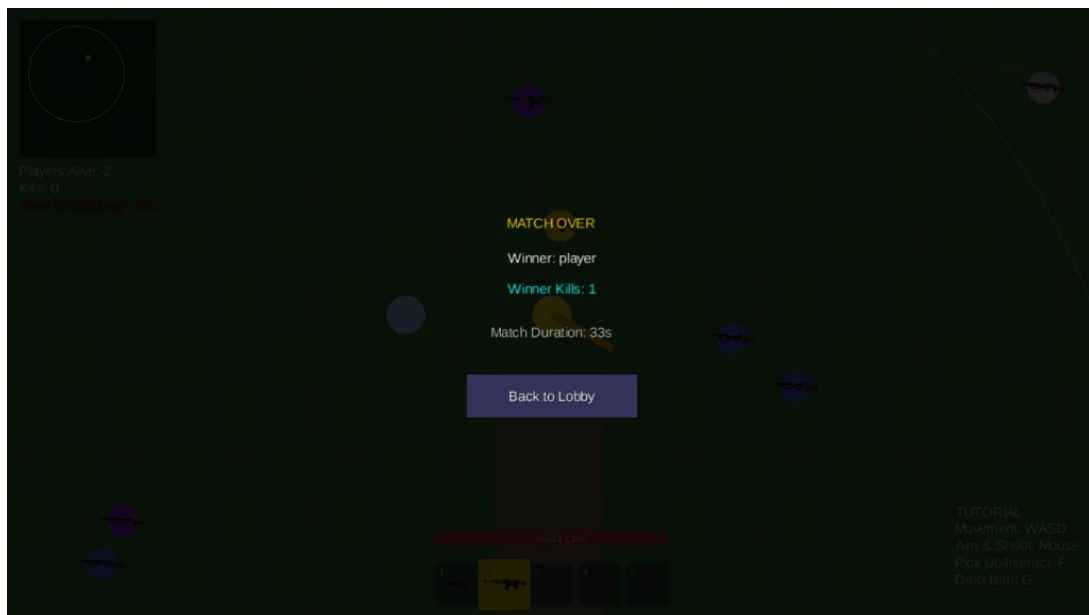
Rysunek 1: Ekran połączenia z serwerem (host, port, nickname)



Rysunek 2: Lobby — lista graczy, status Ready, odliczanie do startu meczu



Rysunek 3: Rozgrywka — HUD, strefa, broń i przedmioty na mapie



Rysunek 4: Ekran końca meczu — zwycięzca, statystyki, powrót do lobby